Notes on Optuna and Hyperopt: Two Interfaces to the Same TPE

Luis Mendez

Overview

Other SMBO Models already derives what TPE optimizes and why: build $\ell(\mathbf{x})$ and $g(\mathbf{x})$ from the good/bad split of observed trials and sample where ℓ/g is large. Both Hyperopt (Section 11) and Optuna (Section 12) ship that same algorithm as `tpe.suggest` / `TPESampler`, and on the breast-cancer XGBoost objective they land on comparable accuracy. What actually differs between the two, and what this note is about, is the *engineering interface* around that shared core: how the search space is declared, how conditional structure is expressed, and what the library does once a trial can be evaluated cheaply at multiple fidelities.

Search-Space Declaration: Define-and-Run vs. Define-by-Run

Hyperopt's `fmin` takes the entire space as one static dictionary built before any trial runs,

```
param_grid = {
  'n_estimators': hp.quniform(
      'n_estimators', 200, 2500, 100),
  'booster': hp.choice(
      'booster', ['gbtree', 'dart']),
  ...
}
```

and `objective(params)` receives a dictionary already shaped like that space; every branch of every `hp.choice` must exist in the dictionary whether or not a given trial ends up using it. Optuna's `objective(trial)` instead calls `trial.suggest_*` imperatively, inline with ordinary Python control flow:

```
classifier_name = trial.suggest_categorical(
    "classifier", ["logit", "RF", "GBM"])
if classifier_name == "logit":
    c = trial.suggest_float("logit_c", 0.001, 10)
    ...
elif classifier_name == "RF":
    n = trial.suggest_int("rf_n_estimators", 100, 1000)
    ...
```

The space is only ever as large as the branch actually taken on a given trial; there is nothing to declare ahead of time for a branch that was not entered. This is the define-by-run design Optuna's own docs advertise, and `02-Conditional-Spaces-Multiple-Models.ipynb` leans on it directly to try three unrelated model families (logistic regression, random forest, GBM) inside one `objective`.

The Cost of Define-and-Run: Conditional Spaces in Hyperopt

`04-Conditonal-Search-Spaces.ipynb` tunes the same three model families in Hyperopt, and the shape of that notebook is the direct consequence of not having define-by-run. The space has to be pre-built as a master dictionary keyed by `hp.choice('model', [...])`, where each branch is itself a nested dictionary carrying the constructor class *and* its parameter sub-space:

```
{'model': LogisticRegression, 'params': {...}}
```

and the objective function unpacks that generically (`params['model']()`, then `params['params']`) rather than branching on a readable classifier name the way the Optuna version does. Worse, because `n_estimators` and `max_depth` only exist for the tree-based branches but the dictionary structure is shared, the int-casting step has to be wrapped defensively,

```
try:
    hyperparams['n_estimators'] = int(hyperparams[
    hyperparams['max_depth'] = int(hyperparams['ma
except:
    pass
```

silently swallowing a `KeyError` on every trial that samples logistic regression. Nothing here is broken — the search still runs and `trials.argmin` still resolves the winner — but a bare `except` on a fixed, foreseeable branch condition is a symptom of the space being declared statically rather than the objective simply not asking for keys it does not need, which is what Optuna's inline `if/elif` does for free.

Live Reporting and Pruning: A Capability Hyperopt Does Not Have

The multi-fidelity idea from *Multi-fidelity Optimization* (successive halving, ASHA, Hyperband as external wrappers around `fit`) has a native, per-trial form in Optuna. Inside the objective, each intermediate fidelity is reported as it becomes available,

```
trial.report(intermediate_accuracy, epoch)
if trial.should_prune():
    raise optuna.TrialPruned()
```

and a `pruner` attached to the study decides, from that stream of reports across all trials, whether the *current* trial is falling behind the pack and should be abandoned before it finishes. `07-successive-halving-optuna.ipynb` uses `SuccessiveHalvingPruner` (`min_resource`, `reduction_factor`, promoting roughly the top third of each "rung" to the next) and `09-hperband.ipynb` uses `HyperbandPruner`, the same underlying policy this course reaches for via `sklearn`'s `HalvingRandomSearchCV` in Section 8, but now driven by the same TPE/random sampler that is also choosing the hyperparameters. Hyperopt has no equivalent hook: `objective(params)` in every Hyperopt notebook in this course returns one final scalar loss with `Trials()` recording only completed trials, so a configuration that is obviously bad after one epoch still runs to completion. Getting successive-halving behavior out of Hyperopt in this course means stepping outside Hyperopt entirely, into `sklearn`'s halving search classes.

Persistence: In-Memory Trials vs. a Storage Backend

Hyperopt's `Trials()` object lives in process memory; `trials.argmin` and `trials.average_best_error()` are convenient for inspecting a run that just finished, but the object does not outlive the Python process on its own. Optuna's `create_study` instead takes a `storage` URL, and `03-Optimizing-a-CNN.ipynb` / `05-Evaluating-the-search.ipynb` point it at a SQLite file (`cnn_study.db` in this section's folder). Because every trial is written through to that file as it completes, the study can be reopened later, resumed with additional trials, or inspected with `study.trials_dataframe()` without having kept the original process alive — which is also the mechanism that makes running independent workers against the same study (parallel search) straightforward, since they only need to agree on the storage URL, not share memory.

What Stayed the Same

It is worth being explicit about what this note is *not* claiming differs: the sampling algorithm. `TPESampler` in Optuna and `tpe.suggest` in Hyperopt are both implementations of the $\ell(\mathbf{x})/g(\mathbf{x})$ density-ratio idea from *Other SMBO Models*; neither library's TPE implementation reasons about pruning or persistence, those are separate concerns layered around the same sampler by each library's engineering choices. Optuna additionally exposes `RandomSampler` and `CmaEsSampler` alongside `TPESampler` as drop-in alternatives selected purely by which object is passed to `create_study`, mirroring how Hyperopt selects `rand.suggest`/`anneal.suggest`/`tpe.suggest` purely by which function is passed to `fmin`'s `algo` argument — the sampler-selection ergonomics are, if anything, the one place the two libraries look almost identical.

Comparing the Interfaces

	Hyperopt	Optuna
Space declaration	Define-and-run (static dict)	Define-by-r
Conditional spaces	Nested dict + `hp.choice`, manual unpacking	Plain `if/e`
Sampler selection	`algo=` in `fmin` (`rand`/`anneal`/`tpe`)	`sampler=`
Pruning / multi-fidelity	None built in	`trial.re`
Persistence	In-memory `Trials()` only	`storage=`
Result inspection	`trials.argmin`, `average_best_error()`	`study.be`

Practical Guidance

- Reach for Optuna when the space has real conditional branching (different model families, architecture search) — `if/elif` on `trial.suggest_categorical` scales far better than pre-building a nested Hyperopt dictionary and defensively unpacking it.
- Reach for Optuna specifically when the objective can report an intermediate score (an epoch, a boosting round) — native pruning turns that into wasted-compute savings for free, which Hyperopt cannot do without leaving the library.
- Hyperopt remains a perfectly reasonable choice for a flat, non-conditional space evaluated to completion every time (the XGBoost breast-cancer objective in Section 11 is exactly this case) — the two libraries' TPE results are not meaningfully different there, so the choice comes down to which interface is already in the codebase.
- If a study needs to survive a process restart, be resumed later, or be attacked by more than one worker process, that alone is reason enough to prefer Optuna's `storage=` over Hyperopt's in-memory `Trials()`.